Lecture 6 — Advanced Geometry: 3D Hulls, Delaunay/Voronoi, Minkowski Sums, Robustness

> World Finals Level · Prerequisites: Year week 31 (geometry I), regional-08 (geometry for regionals), nationals-08 (rotating calipers, half-plane intersection, closest pair) · Estimated time: 2 × 90 min lecture + 3 h lab

1. Overview

Geometry at the World Finals is rarely about an exotic theorem. The ideas behind most WF geometry problems fit on one slide; what separates the teams that solve them is an implementation that is *exactly right* on every degenerate input the judges could think of. This lecture therefore has two intertwined threads:

1. **New algorithms** that appear at the top level and are not covered earlier in the course: the 3D convex hull, the Delaunay triangulation (via the lifting map) and its dual, the Voronoi diagram, and the Minkowski sum of convex polygons.
2. **Robustness as a design goal**: exact predicates with explicit bit budgets, a disciplined split between *predicates* (decisions) and *constructions* (new coordinates), and a testing methodology built around structural checkers and tiny coordinate ranges.

By the end you should be able to write a 3D hull from memory in about 15 minutes, derive Delaunay from it, and reason precisely about when your integer arithmetic can overflow.

2. Robustness: predicates exact, constructions approximate

2.1 The three failure modes

Failure	What happens	Typical cure
Floating-point error	a sign test returns the wrong answer for nearly-degenerate input	integer predicates; carefully scaled epsilon
Degeneracy	collinear/coplanar/duplicate/cocircular input breaks an invariant the code silently assumed	handle explicitly; choose strict vs non-strict comparisons deliberately

Failure	What happens	Typical cure
Overflow	correct on small tests, wrong on large ones	count bits before coding; `__int128`

A geometric algorithm is a combinatorial algorithm driven by *predicates* — questions with a discrete answer such as "is d to the left of ab?" or "is d inside the circumcircle of abc?". If every predicate is answered correctly, the combinatorial output (which faces, which edges) is correct even if later numeric output is slightly off. So: **answer every predicate exactly whenever the input is integral**, and only use floating point to *construct* new points (intersections, circumcentres) at the very end, from original input points.

2.2 Bit budgets

A predicate that is a polynomial of degree d in coordinates of magnitude at most C involves differences of magnitude up to 2C, so its value is bounded by roughly k · (2C)^d where k is the number of terms in the expanded determinant.

- **2D orientation** `cross(b−a, c−a)`: degree 2, two terms. C = 10^9 gives $2 \cdot (2 \cdot 10^9)^2 = 8 \cdot 10^{18} < 9.22 \cdot 10^{18}$ — it *just* fits in `long long`. Any extra term (e.g. comparing two cross products by subtraction) overflows. With C = 10^6 there is plenty of room.
- **3D orientation** (triple product): degree 3, six terms. C = 10^6 gives $6 \cdot (2 \cdot 10^6)^3 \approx 4.8 \cdot 10^{19}$ — **does not fit** in `long long`. Use `__int128` for the products (as in the code below); then C up to $\sim 10^{12}$ is safe.
- **2D incircle**: degree 4. C = 10^6 gives about 2^{86} — `__int128` (2^{127}) is fine; C = 10^9 is at the edge and should be avoided.
- **Lifted orientation** ($x, y, x^2 + y^2$): also degree 4 in the original coordinates; same budget as incircle.

Write the bound in a comment next to the predicate. It takes ten seconds and prevents the most embarrassing WA at the Finals.

2.3 When floating point is unavoidable

If inputs are real numbers, or if the answer is a constructed point, use `long double` and choose epsilon *relative to the magnitude of the operands*: for a cross product of vectors of length up to L, an absolute error of about $L^2 \cdot 10^{-15}$ is realistic, so `eps = 1e-9` is meaningless when L = 10^6. Keep construction chains short (intersect input lines, not lines through previously constructed points). Prefer `atan2(|u×v|, u·v)` over `acos(u·v / (|u||v|))` for angles; the latter returns NaN when rounding pushes the argument slightly above 1 and is badly conditioned near 0 and π.

Symbolic perturbation ("Simulation of Simplicity") can make every input non-degenerate in theory, but it is long and error-prone; at a contest, explicit handling of each degenerate case is almost always the better trade.

3. 3D convex hull

3.1 Facts

For n points in 3D, the hull is a convex polytope. If all faces are triangulated, Euler's formula V − E + F = 2 together with 3F = 2E gives **E ≤ 3n − 6 and F ≤ 2n − 4**. The output therefore has linear size, and O(n log n) is optimal (divide and conquer, or randomised incremental with a conflict graph). In contests the **O(n²) incremental algorithm** is the standard choice: it is short, it is exact with integer coordinates, and it runs in about 0.1 s for 3000 points even in the worst case (all points on a sphere, where every point is a hull vertex).

3.2 The incremental algorithm

1. Find four affinely independent points: p_0, a point different from p_0, a point not collinear with those two, and a point not coplanar with those three. If any search fails, the input is degenerate (all coincident, collinear, or coplanar) and must be handled as a 2D problem.
2. Create the four faces of the tetrahedron. Orient each face outward by checking the fourth vertex: if it is on the positive side, swap two vertices.
3. Insert the remaining points in any order (shuffle for safety). For point p, a face f = (a, b, c) is **visible** if `vol(a, b, c, p) > 0`, i.e. p is strictly outside the plane of f.
4. If no face is visible, p is inside or on the hull — skip it.
5. Otherwise, remove all visible faces. They form a connected region whose boundary is a closed cycle of edges, the **horizon**. For each horizon edge a→b (directed as in the removed face) add the face (a, b, p).

Detecting the horizon. Each face (a, b, c) owns the three directed edges a→b, b→c, c→a. Because the surface is consistently oriented, the neighbour across edge a→b owns b→a. Mark every directed edge of every visible face with a stamp equal to the current step. Then a→b (in a visible face) is a horizon edge iff b→a is *not* stamped in this step. Using the step number as the stamp avoids clearing the table. The new face (a, b, p) inherits the orientation of the deleted face, so it is automatically outward.

3.3 Why strict visibility is right

- If p lies in the plane of face f and *inside* the triangle, p is on the hull surface and adds nothing; it is correctly skipped.
- If p lies in the plane of f but *outside* it, then crossing the nearest edge of f we reach a neighbouring face g that bends away (convexity), so p is strictly above g: some face is visible, and p is added.
- A new face (a, b, p) cannot be degenerate: if p were on the line ab, both faces adjacent to edge ab would have p on their plane, so neither would be strictly visible and a→b could not be a horizon edge.

- Duplicate points are inside, hence skipped.

Using `>= 0` instead breaks these guarantees and creates zero-area faces and inconsistent horizons. The result with `> 0` contains coplanar facets as several triangles; if the problem asks for the number of (polygonal) faces, merge adjacent triangles with coplanar normals afterwards (compare normals exactly: cross products are parallel and point the same way).

3.4 Derived quantities

- Volume: Σ over faces `vol(O, a, b, c) / 6` for any fixed origin O (use a hull vertex to keep numbers small).
- Surface area: Σ |(b − a) × (c − a)| / 2 (this is a construction — use `long double`).
- Centroid of the solid: Σ vol_f · (O + a + b + c)/4 / Σ vol_f.
- Hull edges for "minimum width" or "smallest enclosing box" problems: the antipodal pairs of 3D rotating calipers are rarely required at contests; O(F · n) brute force over face–vertex pairs often suffices.

4. Delaunay triangulation via the lifting map

4.1 The lifting map

Map each plane point (x, y) to $\hat{p} = (x, y, x^2 + y^2)$ on the paraboloid $z = x^2 + y^2$. A circle $(x - u)^2 + (y - v)^2 = r^2$ can be rewritten as $x^2 + y^2 = 2ux + 2vy + (r^2 - u^2 - v^2)$, i.e. as the intersection of the paraboloid with a plane. Points *inside* the circle lift to points *below* that plane. Hence

> d is strictly inside circle(a, b, c) ⇔ $\hat{d}$ is strictly below the plane through $\hat{a}$, $\hat{b}$, $\hat{c}$ ⇔ an orientation test on the lifted points.

Theorem. A triangle abc is Delaunay (its circumcircle contains no input point in its interior) iff $\hat{a}\hat{b}\hat{c}$ is a face of the *lower* convex hull of the lifted points. So the Delaunay triangulation is the projection of the downward-facing faces of a 3D hull.

The same idea one dimension down solves CF 1142C *U2*: parabolas $y = x^2 + bx + c$ become lines under $(x, y) \mapsto (x, y - x^2)$, and "no point strictly above the parabola" becomes "edge of the upper hull".

4.2 Implementation and degenerate inputs

Run the 3D hull on lifted points and keep faces whose outward normal has negative z-component (the z-component of (b−a)×(c−a) is the 2D cross product of the projections). Reverse each kept face to get a counter-clockwise triangle in the plane. Two degenerate inputs make the lifted set coplanar, and the hull routine returns nothing: **all points collinear** (no triangulation exists — the answer is a path) and **all points cocircular** (any triangulation of the

convex polygon is Delaunay). Handle both explicitly. When only some points are cocircular, the 3D hull triangulates the coplanar facet arbitrarily; your answer must not depend on that choice (e.g. EMST is fine because ties do not change the weight).

Bit budget: lifted z is degree 2, the orientation is degree 4; with $|x|, |y| \le 10^6$ everything fits in `__int128`.

4.3 Properties worth memorising

- Among all triangulations it **maximises the minimum angle** (lexicographically, the sorted angle vector).
- It contains the **Euclidean minimum spanning tree** and the **nearest-neighbour graph**. So EMST = Kruskal on $\le 3n - 6$ Delaunay edges.
- It contains the **Gabriel graph** and the **relative neighbourhood graph**.
- Its **dual is the Voronoi diagram**: Voronoi vertices are the circumcentres of Delaunay triangles; two cells share an edge iff the sites are joined by a Delaunay edge.

5. Voronoi diagrams in practice

At contests you rarely need to output the Voronoi diagram; you need an *extremum* that is attained on it. Classic pattern — **largest empty circle**: maximise over points q in a region R the distance from q to the nearest site. The function "distance to nearest site" restricted to one Voronoi cell is the distance to a single point, which is convex, so its maximum over (cell ∩ R) is attained at a vertex of that polygon: a Voronoi vertex inside R, an intersection of a Voronoi edge with ∂R, or a vertex of R. ICPC WF 2018 *Panda Preserve* is exactly this pattern (sites = polygon vertices, region = the polygon).

Two practical constructions:

- **$O(n^2 \log n)$, very robust:** cell of site s = intersection of the half-planes "closer to s than to t" for all $t \ne s$, clipped to a big bounding box (half-plane intersection from nationals-08, or simply clip a convex polygon $n - 1$ times in $O(n^2)$ per cell for small n).
- **Via Delaunay:** compute circumcentres of triangles and connect circumcentres of adjacent triangles. Requires adjacency bookkeeping; unbounded cells need rays for hull edges.

For $n \le 1000$ the clipping approach is usually the right engineering choice.

6. Minkowski sums

6.1 Definition and structure

$A \oplus B = \{ a + b : a \in A, b \in B \}$. If A and B are convex polygons with n and m vertices, $A \oplus B$ is a convex polygon with at most $n + m$ vertices, and its edge sequence (as vectors) is the **merge by polar angle** of the edge sequences of A and B. The vertex of $A \oplus B$ extreme in direction d is the sum of the vertices of A and B extreme in direction d — which is the whole proof.

6.2 Algorithm

Rotate both CCW polygons to start at their lowest (then leftmost) vertex; then both edge sequences start at polar angle in $[0, \pi)$ and are sorted by angle. Walk with two pointers: emit $a[i] + b[j]$, compare the current edges with a cross product, and advance the pointer whose edge has the smaller angle (both when parallel). Parallel edges are merged into one, so no collinear vertices are created as long as the inputs are strictly convex. Remove collinear input vertices first, or the output will contain duplicates.

6.3 Applications

- **Collision detection.** $A \cap B \neq \varnothing \Leftrightarrow 0 \in A \oplus (-B)$. Point-in-convex-polygon is $O(\log n)$ after $O(n + m)$ preprocessing.
- **Distance between convex polygons** = distance from the origin to $A \oplus (-B)$ (0 if inside).
- **Translational motion planning.** A convex robot R translating among obstacles O_i: the reference point is free iff it is outside every $O_i \oplus (-R)$ ("configuration-space obstacles").
- **Sums of many polygons / range queries.** The number of vertices of $P_l \oplus \ldots \oplus P_r$ equals the number of *distinct edge directions* among those polygons. Normalise each edge vector by its gcd, map directions to ids, and the problem becomes offline "distinct values in a range" (CF 1195F).
- **Point in a triple sum.** CF 87E *Mogohu-Rea Idol*: is $3h \in A \oplus B \oplus C$? Build the sum once and answer each query in $O(\log n)$.
- **Maximum of a linear function over $A \oplus B$** separates into maxima over A and over B — sometimes this is all you need and no construction is required.

7. Reference implementations

7.1 3D convex hull: surface area and volume

Input: n, then n lines with integer x y z ($|\text{coord}| \le 10^6$). Output: surface area and volume of the convex hull.

```cpp
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
typedef __int128 LL;

struct P3 {
    ll x, y, z;
    P3 operator-(const P3& o) const { return {x - o.x, y - o.y, z - o.z}; }
};
// exact sign of det[b-a, c-a, d-a]; > 0: d lies on the side the normal (b-a)x(c-a)
points to.
// degree 3: |coord| <= 1e6 needs __int128 (6 * (2e6)^3 ~ 4.8e19)
LL vol(const P3& a, const P3& b, const P3& c, const P3& d) {
    P3 u = b - a, v = c - a, w = d - a;
    return (LL)u.x * ((LL)v.y * w.z - (LL)v.z * w.y)
         - (LL)u.y * ((LL)v.x * w.z - (LL)v.z * w.x)
         + (LL)u.z * ((LL)v.x * w.y - (LL)v.y * w.x);
}

// Incremental 3D hull, O(n^2). Triangles CCW seen from outside; {} if all points
coplanar.
vector<array<int, 3>> hull3d(const vector<P3>& p) {
    int n = p.size();
    if (n < 4) return {};
    int i1 = -1, i2 = -1, i3 = -1;
    for (int i = 1; i < n && i1 < 0; i++)
        if (p[i].x != p[0].x || p[i].y != p[0].y || p[i].z != p[0].z) i1 = i;
    if (i1 < 0) return {};
    for (int i = 1; i < n && i2 < 0; i++) {
        P3 u = p[i1] - p[0], v = p[i] - p[0];
        if ((LL)u.y * v.z - (LL)u.z * v.y != 0 || (LL)u.z * v.x - (LL)u.x * v.z != 0 ||
            (LL)u.x * v.y - (LL)u.y * v.x != 0) i2 = i;
    }
    if (i2 < 0) return {};
    for (int i = 1; i < n && i3 < 0; i++)
        if (vol(p[0], p[i1], p[i2], p[i]) != 0) i3 = i;
    if (i3 < 0) return {};

    vector<array<int, 3>> F;
    vector<vector<int>> stamp(n, vector<int>(n, -1));
    auto addFace = [&](int a, int b, int c, int inside) {
        if (vol(p[a], p[b], p[c], p[inside]) > 0) swap(b, c);
        F.push_back({a, b, c});
    };
    int t[4] = {0, i1, i2, i3};
    for (int k = 0; k < 4; k++) addFace(t[(k + 1) % 4], t[(k + 2) % 4], t[(k + 3) % 4],
t[k]);

    vector<char> used(n, 0);
    for (int k = 0; k < 4; k++) used[t[k]] = 1;
    for (int i = 0; i < n; i++) {
        if (used[i]) continue;
        vector<array<int, 3>> keep, vis;
        for (auto& f : F) (vol(p[f[0]], p[f[1]], p[f[2]], p[i]) > 0 ? vis :
```

```
keep).push_back(f);
        if (vis.empty()) continue;
        for (auto& f : vis)
            for (int e = 0; e < 3; e++) stamp[f[e]][f[(e + 1) % 3]] = i;
        for (auto& f : vis)
            for (int e = 0; e < 3; e++) {
                int a = f[e], b = f[(e + 1) % 3];
                if (stamp[b][a] != i) keep.push_back({a, b, i});
            }
        F.swap(keep);
    }
    return F;
}

int main() {
    int n;
    if (!(cin >> n)) return 0;
    vector<P3> p(n);
    for (auto& q : p) cin >> q.x >> q.y >> q.z;
    shuffle(p.begin(), p.end(), mt19937(20240917));
    auto F = hull3d(p);
    if (F.empty()) { puts("degenerate (coplanar input)"); return 0; }
    long double area = 0, volume = 0;
    for (auto& f : F) {
        P3 u = p[f[1]] - p[f[0]], v = p[f[2]] - p[f[0]];
        long double cx = (long double)u.y * v.z - (long double)u.z * v.y;
        long double cy = (long double)u.z * v.x - (long double)u.x * v.z;
        long double cz = (long double)u.x * v.y - (long double)u.y * v.x;
        area += sqrtl(cx * cx + cy * cy + cz * cz) / 2;
        volume += (long double)vol(p[F[0][0]], p[f[0]], p[f[1]], p[f[2]]) / 6;
    }
    printf("%.6Lf %.6Lf\n", area, volume);
}
```

The `stamp` table is n × n ints (64 MB at n = 4000). For larger n, replace it with an `unordered_map` keyed by `a * n + b`, or store, for each face, the indices of its three neighbours.

7.2 Delaunay triangulation (reuses `P3`, `vol`, `hull3d` above)

```
// Requires: distinct points, not all collinear, not all cocircular, |coord| <= 1e6.
vector<array<int, 3>> delaunay(const vector<pair<ll, ll>>& pts) {
    vector<P3> p;
    for (auto [x, y] : pts) p.push_back({x, y, x * x + y * y});
    vector<array<int, 3>> res;
    for (auto& f : hull3d(p)) {
        P3 u = p[f[1]] - p[f[0]], v = p[f[2]] - p[f[0]];
        if ((LL)u.x * v.y - (LL)u.y * v.x < 0) res.push_back({f[0], f[2], f[1]});
    }
    return res;
}
```

We validated this against a brute-force checker on thousands of random inputs with coordinates in $[-3, 3]$ (dense degeneracies): every triangle is CCW, no point is strictly inside any circumcircle (exact incircle determinant), and the triangle areas sum to the hull area.

7.3 Minkowski sum and convex-polygon collision

Input: two convex polygons (CCW, no three collinear consecutive vertices). Output whether they intersect (touching counts), via $0 \in A \oplus (-B)$.

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;

struct P {
    ll x, y;
    P operator+(P o) const { return {x + o.x, y + o.y}; }
    P operator-(P o) const { return {x - o.x, y - o.y}; }
    ll cross(P o) const { return x * o.y - y * o.x; }   // |coord| <= 1e9 after summing:
ok to ~2e9
};
void reorder(vector<P>& a) {
    int k = 0;
    for (int i = 1; i < (int)a.size(); i++)
        if (a[i].y < a[k].y || (a[i].y == a[k].y && a[i].x < a[k].x)) k = i;
    rotate(a.begin(), a.begin() + k, a.end());
}
vector<P> minkowski(vector<P> a, vector<P> b) {
    reorder(a); reorder(b);
    int n = a.size(), m = b.size();
    a.push_back(a[0]); a.push_back(a[1]);
    b.push_back(b[0]); b.push_back(b[1]);
    vector<P> r;
    int i = 0, j = 0;
    while (i < n || j < m) {
        r.push_back(a[i] + b[j]);
        ll c = (a[i + 1] - a[i]).cross(b[j + 1] - b[j]);
        if (c >= 0 && i < n) i++;
        if (c <= 0 && j < m) j++;
    }
    return r;
}
// point in convex polygon (CCW, r[0] lowest-leftmost), boundary counts as inside, O(log
n)
bool inConvex(const vector<P>& r, P q) {
    int n = r.size();
    if (n < 3) return false;
    if ((r[1] - r[0]).cross(q - r[0]) < 0 || (r[n - 1] - r[0]).cross(q - r[0]) > 0)
return false;
    int lo = 1, hi = n - 1;                      // find wedge r[0], r[lo], r[lo+1]
    while (hi - lo > 1) {
        int mid = (lo + hi) / 2;
        if ((r[mid] - r[0]).cross(q - r[0]) >= 0) lo = mid; else hi = mid;
```

```
    }
    return (r[lo + 1] - r[lo]).cross(q - r[lo]) >= 0;
}

int main() {
    int n, m;
    cin >> n;
    vector<P> a(n);
    for (auto& q : a) cin >> q.x >> q.y;
    cin >> m;
    vector<P> b(m);
    for (auto& q : b) cin >> q.x >> q.y, q = P{-q.x, -q.y};   // -B is still CCW
    vector<P> s = minkowski(a, b);
    puts(inConvex(s, P{0, 0}) ? "intersect" : "disjoint");
}
```

The Minkowski routine was tested against "convex hull of all n·m pairwise sums" on 20,000 random polygon pairs.

8. Common mistakes

1. **Overflow in a degree-3 or degree-4 predicate.** `long long` triple products with coordinates 10^6 overflow. Do the multiplications in `__int128`.
2. **Non-strict visibility** in the 3D hull (`>= 0`): creates degenerate faces, double-counts horizon edges, and corrupts the surface.
3. **Forgetting the degenerate start**: no four non-coplanar points exist. The hull routine must report this rather than loop or crash; the caller then solves the 2D problem.
4. **Comparing normals with floating point** when merging coplanar triangles; compare integer cross products for parallelism instead.
5. **Delaunay on cocircular or collinear input**: the lifted points are coplanar and the 3D hull is empty.
6. **`acos` of a value slightly above 1** → NaN. Clamp, or use `atan2(|u × v|, u · v)`.
7. **Minkowski with collinear input vertices** → duplicate points in the sum; point-in-polygon binary search then misbehaves. Clean the polygons first.
8. **Mixing predicates and constructions**: computing an intersection point in `double` and then feeding it to an exact orientation test reintroduces rounding at exactly the place you tried to avoid it.
9. **Not testing on tiny coordinate ranges.** Random coordinates in $[-10^9, 10^9]$ almost never produce degeneracies; $[-2, 2]$ produces them constantly.

9. Practice problems

Problem	Technique
Luogu P4724 — 【模板】三维凸包	3D hull, surface area

Problem	Technique
SPOJ CH3D — 3D Convex Hull	3D hull area and volume
HDU 3662 — 3D Convex Hull	3D hull, count polygonal faces (merge coplanar triangles)
CF 1142C — U2	lifting transform, upper hull
CF 87E — Mogohu-Rea Idol	Minkowski sum of three polygons, point in convex polygon
CF 1195F — Geometers Anonymous Club	Minkowski sum vertex count = distinct directions, offline
ICPC WF 2018 — Panda Preserve (Kattis `pandapreserve`)	Voronoi / largest empty circle in a polygon
ICPC WF 2015 — Asteroids (Kattis `asteroids`)	convex polygon intersection area over time
ICPC WF 2017 — Airport Construction (Kattis `airport`)	robust segment-inside-polygon tests

Suggested lab order: Luogu P4724 (hull warm-up) → CF 87E → CF 1142C → Panda Preserve → CF 1195F.

10. Further reading

- M. de Berg, O. Cheong, M. van Kreveld, M. Overmars, *Computational Geometry: Algorithms and Applications*, 3rd ed. — chapters 9 (Delaunay), 11 (3D hulls), 13 (Minkowski sums / motion planning).
- J. R. Shewchuk, "Adaptive Precision Floating-Point Arithmetic and Fast Robust Geometric Predicates" (1997) — the standard reference on exact predicates.
- H. Edelsbrunner, E. Mücke, "Simulation of Simplicity" (1990) — symbolic perturbation.
- KACTL (KTH team notebook), `content/geometry/` — compact 3D hull and Delaunay implementations: https://github.com/kth-competitive-programming/kactl
- cp-algorithms: "Minkowski sum of convex polygons" and "Delaunay triangulation and Voronoi diagram": https://cp-algorithms.com/